Achievements and Challenges of Metamorphic Testing

Zhan-wei Hui, *Student Member, IEEE/ACM/CCF; Song Huang, Member, IEEE/ACM/CCF;*
Software Testing and Evaluation Centre, PLA University of Science and Technology
Nanjing, Jiangsu Province, China.

***Abstract*—A fundamental problem software testing faces is the so called "test oracle problem", which restricts the development of software testing techniques significantly. Metamorphic Testing (MT) can provide a way to determine the correctness of testing outputs with metamorphic relations between a series of outputs that correspond to a series of inputs, which is regarded as one of the most effective techniques to alleviate oracle problem. This paper reviews previous works on MT and proposes the major challenges of MT, with the goal of supporting future practices and researches in this area.**

Keywords- test oracle problem; metamorphic testing; metamorphic relation

I. INTRODUCTION *(HEADING 1)*

Although software testing cannot guarantee the absence of faults, it is the most popular approach to reveal software defects [5]. One fundamental problem of software testing is the so called "Test oracle problem". Test oracle [39] is a mechanism for checking the correctness of testing results. An ideal test oracle could give "pass/no pass" judgments for test data [7]. Unfortunately, an ideal oracle is either hard to obtain or difficult to apply [33]. For instance, the oracle of numerical analysis software such as programs on partial differential equations is really hard to obtain [27]. In testing of GIS software, engineers are hard to judge the outputs of spatial distance calculation without test oracle [22].

However, even though the ideal oracle is hard to get sometimes, testing can still be carried out. By identifying the necessary relations among outputs of the target program and checking whether the actual outputs satisfy these relations, the correctness of the program would get enhanced to some degree, and confidence to the correctness of programs could also be increased, such as necessary properties [7], data diversity [1], Program checkers and self-testing [12]. By expanding these techniques, T.Y.Chen [28] proposed a kind of software testing technique based on metamorphic relations (MR), called metamorphic testing (MT), which checks the relation between inputs and outputs to determine whether the program satisfies the necessary properties. As a result, it will be unnecessary to assume the existence an oracle for individual inputs, and hence the test oracle problem can be alleviated. The necessary properties of the program under test are called MR.

In this paper, we review on the state of MT and provide the underlying challenges of MT. The remaining part of this paper is organized as follows: Section Ⅱ, we introduce the basic conceptions of MT and MR. Section Ⅲ, we classify the researches on MT into the following categories: MR metric and selection, MR generation and automatic testing, Original test cases generation and test coverage, test evaluation and validation, and the application of MT. For each of the categories, we present the research motivation, different solutions, and current state of the research aspect. Section Ⅳ, we propose the potential problems of current MT researches, and recommend future research directions of MT. Finally, in the last section, we conclude the paper.

II. METAMORPHIC RELATION

Suppose program P is the implementation of function f, and $x_1, x_2, \ldots, x_n (n>1)$is n groups of input for f, their corresponding outputs are $f(x_1), f(x_2), \ldots, f(x_n)$. If $x_1, x_2, \ldots, x_n$ satisfy relation r, it can be referred that $f(x_1), f(x_2), \ldots, f(x_n)$ satisfy relation r_f, that is:

$$r(x_1,x_2,\ldots,x_n)\Rightarrow r_f(f(x_1), f(x_2),\ldots,f(x_n)) \quad (2\text{-}1)$$

Then (r, r_f) is called a metamorphic relation of P [28].

Therefore, if P is correct, then it must satisfy the following comprehension:

$$r(I_1,I_2,\ldots,I_n)\Rightarrow r_f(P(I_1), P(I_2), \ldots, P(I_n)) \quad (2\text{-}2)$$

$I_1, I_2, \ldots, I_n$ are actual inputs of P corresponding with $x_1, x_2, \ldots, x_n$ and $P(I_1), P(I_2), \ldots, P(I_n)$ are the outputs. People could verify the correctness of P by checking whether expression (2-2) is satisfied while testing.

Generally speaking, the input of function f is the same as the input of program P. Therefore, we use $x_1, x_2, \ldots, x_n$ to represent the input in this paper. So if the outputs of test cases don't satisfy the above formula, then the hypothesis is wrong and there are faults in the program.

Based on the previous definition, a formal definition of MR is proposed [10], which expresses the relation by symbolic logic as

$$\text{MR}: \frac{P(x_i)=[P](x_i)}{r(x_1,x_2,..,x_n)\rightarrow r_f([P](x_1), [P](x_2),...,[P](x_n))} (i=1,2,...,n)$$

Then, the relation r is called as Input Relation of Metamorphic Relation (MR_IR), which denote as $IR=\{(x_1, x_2, \ldots, x_n)|r(x_1, x_2, \ldots, x_n)\}$. The relation r_f is called as Output Relation (MR_OR), which also denote as $OR=\{([P](x_1), [P](x_2), \ldots, [P](x_n))| r_f([P](x_1), [P](x_2), \ldots, [P](x_n))\}$. And the relation [P](x) is called as Self Relation of Metamorphic Relation. Therefore, metamorphic relation could be repressed as MR=[IR, SF, OR] [10].

III. THE STATE OF MT RESEARCH

MT has achieved many important accomplishments on account of work done by previous researches. This section is going to review these researches by different research

978-1-4799-2883-5/13 $31.00 © 2013 IEEE
DOI 10.1109/WCSE.2013.16

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:53:58 UTC from IEEE Xplore. Restrictions apply.

issues in detail, which would be useful to get the research focus of metamorphic testing.

A. Construction of MRs

One of the key problems of MT is "how to construct proper metamorphic relations for SUT" [40]. C. Murphy and G. Kaise pointed out that there are three main challenges. First, lots of man-made MRs are needed for MT [14]. Current researches of MT focus on experimental instances, where most of the MRs are constructed by manual work. Second, most of the input data of MT is very complicated and is not human-readable format. Besides, the transform and calculation of a large amount of data is tedious and trivial, which will possibly lead to a wrong conclusion. Third, the situation that "fake errors" exist in the comparison conclusion happens from time to time, it is also very difficult to eliminate those "fake errors".

To solve these problems above, Gotlieb [35] proposed the notion of Automated Metamorphic Testing (AMT) for the first time, the authors focused on the automated generation of input data that could probably arouse the dissatisfaction of MRs, rather than the automated verification of the satisfaction of these relations. C. Murphy and his fellows proposed an Automated Metamorphic System Testing [4], which enables the automated testing for the global MRs of the SUT. To promote the automation of MT and its fault-detecting ability, Murphy [6] realized an automated testing tool called Amsterdam. By testing method, the uncertainty caused by manual work could be reduced as much as possible. However, this method requires that the global MRs have been determined manually before the testing starts, which means the automation of the construction of MRs is unrealized and needs further study. Given the researches done above, we may find that the automation of MT mainly focused on the automated execution of test cases and the verification of MRs. However, the researches and the achievements still could not solve the problem of "how to construct proper metamorphic relations for SUT" [40] thoroughly, meanwhile, it is nearly impossible to obtain metamorphic relations that could be applied in a wide range of fields according to the experiences.

TABLE I. CLASSES OF METAMORPHIC RELATIONS OF NUMERICAL PROGRAMS

No.	Name	Transform of original input
1	Additive	Increase (or decrease) numerical values by a constant
2	Multiplicative	Multiply numerical values by a constant
3	Permutative	Permute the order of elements in a set
4	Invertive	Take the inverse of each element in a set
5	Inclusive	Add a new element to a set
6	Exclusive	Remove an element from a set
7	Compositional	Create a set from some number of smaller sets

Considering theses problems, Murphy [40] proposed 7 common metamorphic properties for numerical programs, which are listed in Table I. Although the classification did not include all the necessary properties of the program, it represented some of the basic mathematical properties of numerical programs, such as distributivity, transitivity and combinability.

However, the application of the above metamorphic relations is limited. When it comes to the testing other kinds of programs such as decision supportive systems, supervision and classifier programs, and random optimization programs, these concluded metamorphic relations may lose effect.

Therefore, Murphy proposed a widely applied method for constructing MRs in his doctoral dissertation [40]. First, consider the metamorphic properties of all applications in the given domain. Next, consider the properties of the algorithm chosen to solve a particular problem in that domain. Then, consider the properties specific to the implementation of the algorithm used to solve the problem. Last, consider properties that are applicable only to the given input that is being used as part of the test case.

The procedure described above works effectively in constructing metamorphic relations for many kinds of programs. But it is still lack of a practical procedure for the execution of the method, and manual work is necessary for the construction of metamorphic properties of every level. By a case study, [41] creatively proposed a method to obtain composite MRs on the basis of known relations, and proved composite metamorphic relations' high fault detection ratio (FD) with two case studies. Besides, it needs further studies.

B. Selection of MRs

The key procedure of MT is to determine effective metamorphic relations. T.Y. Chen suggests that theoretically a stronger relation doesn't mean higher failure-revealing ability according to the experimental results [32]. Even for the same SUT, MRs constructed by different testers would not be the same due to their specific understanding of SUT, so we can also say that the construction of MRs is undetermined. Besides the order of MRs used also affects the testing results, so how to pick out better metamorphic relations deserves much attention.

Mayer [15] suggested that typically good MRs usually contained much of the semantics of the SUT. Based on the experiment results, the following general rules for judging the suitability of MRs are derived: equalities, equalities of linear combinations, semantically rich relations and close to common implementations/algorithms. At last, they thought that composition of some other relations could conquer the problem of using a single MR, but would require higher cost. Besides, the specific steps of metamorphic relation construction weren't given in the paper.

Chen [29] concluded that MRs which could make the multiple executions of the program different would perform better in detecting software defects. Chen et a1. also proposed that theoretical knowledge of the problem domain

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:53:58 UTC from IEEE Xplore. Restrictions apply.

was not adequate for distinguishing good MRs, and good MRs should be those that could make the multiple executions of the software under test (SUT) as different as possible. Therefore, it was proposed that good MRs should be selected with regard to the algorithm that the SUT follows.

Based on the researches in the above two papers, Asrafi [3] studied the relationship between the executions caused by MR, which found that MRs with higher code coverage mostly did better in detecting software defects, but not all of them. Chan [20] did a further study about the selection strategy of metamorphic relations based on Chen's conclusion of "good MRs should be those that can make the original executions and the follow-up executions as different as possible", and proposed three more detailed guidelines for the selection of metamorphic relations. However, the results are not adequate enough and their realization needs the hypothesis that the path execution information is given.

Dong [41] studied the selection strategy and effectiveness evaluation of metamorphic relations on the basis of Chen's researches [29], and then proposed three guidelines for devising metamorphic relations. The three guidelines are more practical than those proposed before, however, there are still several problems left such as, how to measure the complexity of metamorphic relations, how to determine the number of variables in the input of the program and how to determine the coverage of metamorphic relations. All of these bring uncertainty into the selection ofMRs.

These conclusions help testers a lot in selecting metamorphic relations, but they are still just qualitative, which would make the practical selection of MR lack of objective bases.

C. Generation of Original Test Cases

Construction of MRs and original test cases are the two key procedures in MT, and the latter plays a fundamental role, which means the quality of original test cases affects the testing efficiency greatly. Usually, the input field is infinite, when it is impossible to do traverse test [31]. So, people have come up with a bunch of test case selection strategies, in which random test case generation and special test case generation are the most popular ways, each of them has its own advantages.

Special testing method is a simple and practical way, which is widely used in software testing. However, special testing doesn't do well enough in detecting faults in software compared to other conventional testing methods. Random testing has a higher fault-detecting ratio compared with special testing and was proposed for dependability testing at first. Wu [18] made a case study about the test case generation techniques of metamorphic testing with a real program of sparse matrix multiplication, which was a part of the spare matrix multiplication benchmark JASPA. Experimental results showed metamorphic testing exhibits better performance with random test cases on account of two measurements: mutation score and fault detection ratio.

One universal problem they share is the test coverage problem. On the one hand, all of the three methods would produce a large amount of "resemble" test cases [25]. On the other hand, it may happen that some of the paths are covered repeatedly while some others are omitted and left untested. To solve the problem, Dong et a1 [8] proposed three metamorphic testing criteria on the basis of path coverage criteria. The authors also presented three new testing algorithms for generating test suites that could satisfy the three criteria above in [8]. Experimental results showed that the three algorithms could detect faults quickly and exactly with fewer test cases than traditional methods.

D. Metamorphic Testing Effectiveness

The efficiency of MT decides the quality of the test, so an efficient way of evaluation is needed, and the results can also help improving our test methods. However, most of the time testers have little knowledge about the existed faults in the program under test, so it is pretty hard to evaluate the performance of metamorphic testing method.

Currently, one of the most popular and effective way to evaluate the efficiency of metamorphic testing is the mutation analysis technique [2], which was originated from the fault-based testing technique. On testing efficiency of metamorphic testing, [3] presented corresponding measurements based on the fault/mutate detection probability and consistency of MR. [3] presented the definitions of MR probability (MRP) for the first time. Fault detection probability within a range (FDPR) and fault detection probability (FDP) show as follows:

$$\mathrm{MRP} = \frac{\text{Number of MRs in the range}}{\text{Total Number of MRs for SUT}} \times 100\%$$

which is the probability of a MR within a range.

$$\mathrm{FDPR} = \frac{\text{Number of Fault Detecting MRs in the range}}{\text{Total Number of MRs in the range}} \times 100\%,$$

which is the fault detection of a MR within a range.

FDP=MRP*FDPR, which is the fault detection of all MRs within a range.

Wu [18] presented two measurements for the quality of a test set: mutation score (MS), which is the percentage of mutants detected and used to measure the quality of a test case , and fault detection ratio (FD), which is the percentage of test cases that could detect certain mutant.

As we can see, the two evaluation indicators of efficiency above focus on the relationship between metamorphic relation and mutant without inspecting the time of testing.

However, the problem of this evaluation method lies in the uncertainty of the time. Besides, the time indicator "Smallest Observation" given by [13] wasn't defined clearly enough, which may affect the results of the efficiency evaluation of metamorphic testing, and the feasibility and creditability of the evaluation are also doubtful. MS and FD are only suitable for comparing the

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:53:58 UTC from IEEE Xplore. Restrictions apply.

effectiveness of metamorphic testing and other testing techniques, such as random testing and special testing, while MRP, FDPR and FDP are able to estimate the efficiency of several MRs in metamorphic testing.

IV. The Challenges of Metamorphic Testing

Based on the researches people have done in MT, we find that most of the achievements lie in its application with few researches in the theoretical areas. Although there have been several academic teams for metamorphic testing, who have solved a few important problems in this technique, there are still several technical issues needing solutions, with which the development of not only MT technique but also the other software testing techniques will be promoted greatly.

A. Soundness and Correctness of MRs: Lack of formalized description methods for metamorphic relations with consistency and without ambiguity

As so many kinds of metamorphic relations are constructed in different research papers [16] [37] [24] [36] [21] [17] [20] [23] [26] [30] [92] [34] [9] [19] [11], and they are described in different ways, the soundness and correctness of MRs becomes more and more critical for metamorphic testing, which means that how do we know an MR is correct. This brings the problem that metamorphic relations proposed in different research papers are ambiguous, which makes it hard to verify the completeness of the relations and to reuse them.

B. Effectiveness and Adequacy of MRs: Lack of objective and efficient metrics for metamorphic relations

Effectiveness and adequacy of MRs is another issue for MT researches. Efficient metrics of MRs are important for optimization and selection. However, current metrics given by the existed researches from [38] [29] [3] [18] [13] are usually qualitative but not quantitative. Besides, these metrics are not semantically clear, which affects their application in selection and comparison of metamorphic relations. However, the traditional software metrics may not suitable for this and their efficiency need further studies. So here, we are going to make a deep research on efficient metrics for MRs. According to our preliminary researches, we find that lots of variables and mathematic relations need to be simplified when metamorphic relations are described by logic language. This is quite important for further metric of metamorphic relations. Actually, metamorphic relations can be divided into several sub-relations by analysis.

C. Construction and Completeness of MRs: Construct more efficient MRs based on basic ones to ensure the completeness of MRs

Completeness of MRs is to answer the question that how do we know we have found all the MRs for SUT. And first of all, we should have a kind of effective method which could construct MRs. The construction of metamorphic relations needs creativity and is lack of criteria, which not only restricts the automation of MT, but also limits the completeness of MRs.

V. Conclusions

MT regards as one of most effective one up to the present, and has been widely studied and applied in many fields to alleviate test oracle problem. With presented researches, MT has provided many experiences in solving the test oracle problem. However, there are still several technical issues needing solutions, which will promote the development of not only metamorphic testing technique but also the other software testing techniques greatly.

Acknowledgments

This work was supported by National High Technology Research and Development Program of China Project (No: 2009AA01Z402), the Natural Science Foundation of Jiangsu Province, China (No: BK2012059, BK2012060).

References

[1] Ammann, P. E. and Knight, J.C. Data diversity: an approach to software fault tolerance, IEEE Transactions on Computers 1988, 37, 4, 418–425.

[2] Andrews, J. H., Briand, L. C., and Labiche, Y. Is mutation an appropriate tool for testing experiments? ICSE 2005, ACM, New York, NY, USA, 402-411.

[3] Asrafi, M., Liu, H., and Kuo, F.C. On Testing Effectiveness of Metamorphic Relations: A Case Study. 2011 Fifth International Conference on Secure Software Integration and Reliability Improvement. 2011, IEEE Computer Society Washington, DC, USA, 147-156.

[4] Murphy, C., Shen, K., and Kaiser, G. Automatic system testing of programs without test oracles. In Proceedings of the Eighteenth international Symposium on Software Testing and Analysis (ISSTA 2009), Chicago, Illinois, USA, 2009, 189-199.

[5] Beizer, B. 1990. Software Testing Techniques. New York, NY: Van Nostrand Reinhold.

[6] Murphy, C., Kaiser, G., Vo, I., and Chu, M. Quality assurance of software applications using the in vivo testing approach. In Proc. of the Second IEEE International Conference on Software Testing, Verification and Validation (ICST'2009), Denver, Colorado, USA, 2009, 111–120.

[7] Baresi, L. and Young, M. Test oracles, Technical Report CIS-TR01-02, Department of Computer and Information Science 2001, University of Oregon, Eugene, OR.

[8] Dong, G.W., Nie, C.H., Xu, B.W., AND WANG, L.L. An effective iterative metamorphic testing algorithm based on program path analysis. Proceeding of 7th Annual International Conference on Quality Software (QsIc 2007), 2007, 11-October 12, Portland, Oregon, USA, 292—297.

[9] Murphy, C., Kaiser, G., Hu, L., and Wu, L. Properties of machine learning applications for use in metamorphic testing. In Proc. of the 20th International Conference on Software Engineering and Knowledge Engineering (SEKE'2008), San Francisco, USA, 2008, 867–872.

[10] Hui, Z.W., Huang, S., and Ren Z.P. Metamorphic Testing Integer Overflow Faults of Mission Critical Programs: A Case Study. Mathematical Problems in Engineering. Volume 2013, Article ID 381389, 2013, http://dx.doi.org/10.1155/2013/381389.

[11] Xie, X.Y., W.K.Ho, J., Murphy, C., Kaiser, G., Xu, B.W., and Chen, T.Y. Testing and Validating Machine Learning Classifiers by Metamorphic Testing. Journal of Systems and Software, 84 4, April, 2011, 544-558.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:53:58 UTC from IEEE Xplore. Restrictions apply.

[12] Blum, M. and Kannan, S. 1995. Designing programs that check their work, Proceedings of the 31st Annual ACM Symposium on Theory of Computing (STOC '89), 86–97. New York, NY: ACM Press, 1989. Also Journal of the ACM, 1995, vol. 42, no. 1, 269–291.

[13] Zhang, Z.Y., Chen, W.K., and Tse, T.H., Hu, P.F. Experimental Study to Compare the Use of Metamorphic Testing and Assertion Checking. Journal of Software, 20, 10, 2009, 2637–2654.

[14] Murphy, C. and Kaiser, G. Metamorphic Runtime Checking of Non-Testable Programs. Columbia University Dept of Computer Science Tech Report cucs-042-09, 2009, http:// hdl.handle.net /10022/ AC:p: 9293.

[15] Mayer, J., and Guderlei, R. An Empirical Study on the Selection of Good Metamorphic Relations. In Proceedings of the 30th Annual international Computer Software and Applications Conference (COMPSAC 2006), Chicago, IL, 2006, 475-484.

[16] Zhou, Z.Q., Tse, T.H., Kuo, F.C., and Chen, T.Y. Automated functional testing of Web search engines in the absence of an oracle, Technical Report TR-2007-06, Department of Computer Science, The University of Hong Kong, Hong Kong, 2007.

[17] Chan, W.K., C. F. Ho, J., and Tse, T.H. Piping Classification to Metamorphic Testing: An Empirical Study towards Better Effectiveness for the Identification of Failures in Mesh Simplification Programs. COMPSAC, City University of Hong Kong, Kowloon, China, 2007, 1, 397-404.

[18] Wu, P., Shi, X.C., Tang, J.J., Lin, H.M., and Chen, T.Y. Metamorphic testing and special case testing : A case study, Journal of Software, 16, 7, 2005, 1210—1220.

[19] Xie, X.Y., W.K.Ho, J., Murphy, C., Kaiser, G., Xu, B.W., and Chen, T,Y. Application of metamorphic testing to supervised classifiers. In Proc. of the 9th International Conference on Quality Software (QSIC 2009), Jeju, South Korea, 2009, 135-144.

[20] Chan, W.K., Chen, T.Y., Lu, H., Tse, T.H., and Yau, S.S. A metamorphic approach to integration testing of context-sensitive middleware-based applications. In Proceedings of the Fifth International Conference on Quality Software (QSIC 2005), Los Alamitos, California. IEEE Computer Society Press 2005, 241–249.

[21] Chan, W.K., Chen, T.Y., Lu, H., Tse, T.H., and Yau, S. S. Integration testing of context-sensitive middleware-based applications: a metamorphic approach, International Journal of Software Engineering and Knowledge Engineering, 2006, 16, 5, 677–703.

[22] Hui, Z.W. and Huang, S. 2012. Research on Fundamental of Metamorphic Testing. 2012 PLA Graduate of Engineering Innovation Forum (PLAGEIF) (Best Paper Award), Wuhan, China 2012.11, 217-223 (In Chinese).

[23] Chan, W.K., Cheung, S.C., and Leung, K. R. P.H. A metamorphic testing approach for online testing of service-oriented software applications, International Journal of Web Services Research, 2007, 4, 2, 60–80.

[24] Sun, C.A., Wang. G., Mu, B.H., Liu, H., Wang, Z.S., and Chen, T. Y. A Metamorphic Relation-Based Approach to Testing Web Services without Oracles. International Journal of Web Services Research (IJWSR), 9, 1, 2012, 51-73.

[25] Dong, G.W., Nie, C.H., and Xu, B.W. Effective metamorphic testing based on program path analysis. Chinese Journal of Computers, 2009, 32 5. 1002-1013. (In Chinese)

[26] Zhou, Z.Q., Zhang, S., Hagenbuchner, M., Tse, T.H., Kuo, F.-C., and Chen, T.Y. Automated Functional Testing of Online Search Services, Software Testing, Verification and Reliability, 2010, doi:10.1002/stvr.437.

[27] Chen, T.Y., Feng, J., and Tse, T.H. Metamorphic testing of programs on partial differential equations: a case study, Proceedings of the 26th Annual International Computer Software and Applications Conference (COMPSAC 2002). IEEE Computer Society Press, Los Alamitos, 2002, 327–333.

[28] Chen, T.Y., Cheung, S.C., and Yiu, S.M. Metamorphic testing: a new approach for generating next test cases, Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, Hong Kong, 1998.

[29] Chen, T. Y., Huang, D. H., Tse, T. H., and Zhou, Z. Q. Case studies on the selection of useful relations in metamorphic testing. In Proceedings of the 4th Ibero-American Symposium on Software Engineering and Knowledge Engineering, Madrid, Spain. Polytechnic University of Madrid, 2004, 569–583.

[30] Lu, X.L, and Dong, Y.W. Testing of service-oriented software: a metamorphic testing approach. Journal of Computer Applications, 2011,31. 7, 1756-1758.

[31] Chen, T.Y., Kuo, F.C., Liu, Y., and Tang, A. Metamorphic testing and testing with special values. Proceeding of the 5th International Conference on Software Engineering, Artificial Intelligence, Networking, and Parallel/Distributed Computing(SNPD 2004), Mt. Pleasant, Michigan, 2004, 128-134.

[32] Chen, T.Y., Kuo, F.C., Tse, T.H., Zhou, Z.Q. Metamorphic Testing and Beyond. Proceedings of the International Workshop on Software Technology and Engineering Practice (STEP 2003), Los Alamitos: IEEE Computer Society Press, 2003, 94-100.

[33] Chen, T.Y., Kuo, F.-C., Tam, W.K., and Merkel, R. Testing a software based PID controller using metamorphic testing, Proceedings of the International Conference on Pervasive and Embedded Computing and Communication Systems. PECCS2011, 2011, 387-396.

[34] Murphy, C., Kaiser, G., and Arias, M. An approach to software testing of machine learning applications. In Proc. of the 19th international conference on software engineering and knowledge engineering (SEKE'2007), Boston, Massachusetts, USA, 167–172, 2007.

[35] Gotlieb, A. and Botella, B. Automated metamorphic testing. In Proceedings of the 27th IEEE Annual International Computer Software and Applications Conference (COMPSAC2003), November, 2003, 34–40.

[36] Sun, C.A., Wang. G., Mu, B.H., Liu, H., Wang, Z.S., and Chen, T. Y. Metamorphic Testing for Web Services: Framework and a Case Study. 2011 IEEE International Conference on Web Services (ICWS), Washington, DC, 4-9 July 2011, 283 – 290.

[37] Lu, X.L., Dong, Y.W., and Luo, C. Testing of Component-based Software: a Metamorphic Testing Methodology. 2010 7th International Conference on Ubiquitous Intelligence & Computing and 7th International Conference on Autonomic & Trusted Computing (UIC/ATC), 26-29 Oct, 2010, Xian, Shaanxi, 272-276.

[38] Mayer, J. and Guderlei, R. An empirical study on the selection of good metamorphic relations, Proceeding of the 30th Annual International Computer Software and Applications Conference (COMPSAC2006), 2006, 475-484.

[39] Weyuker, E. J. On testing non-testable programs, The Computer Journal, 25, 4, 1982, 465–470.

[40] Murphy, C., Vaughan, M., Ilahi, W., and Kaiser, G. Automatic detection of previously-unseen application states for deployment environment testing and analysis. In Proc. of the 5th International Workshop on Automation of Software Test (AST), Cape Town, South Africa, 2010, 16-23.

[41] Dong, G.W., Xu, B.W., Chen, L, Nie, C.H., and Wang, L.L. Case Studies on Testing with Compositional Metamorphic Relations. Journal of Southeast University (English Edition), 2008, 24, 4, 437 - 443.

[42] Murphy, C. and Kaiser, G. Improving the dependability of machine learning applications. Technical Report CUCS-49-08, Dept. of Computer Science, Columbia University, 2008.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:53:58 UTC from IEEE Xplore. Restrictions apply.